

MinimumEigenOptimizer returns a wrong solution

<https://github.com/qiskit-community/qiskit-optimization/issues/92>

ROW 102

MinimumEigenOptimizer returns a wrong solution #92

New issue

Closed

#92



t-imamichi opened on Apr 21, 2021

Information

- Qiskit Optimization version: latest
- Python version: 3.8.9
- Operating system: macOS 11.2.3

What is the current behavior?

MinimumEigenOptimizer with qasm_simulator returns a wrong solution.

Steps to reproduce the problem

```
from docplex.mp.model import Model
mdl = Model("docplex model")
x = mdl.binary_var('x')
y = mdl.binary_var('y')
mdl.minimize(x-2*y)
print(mdl.export_as_lp_string())
result = mdl.solve()
print("result of CPLEX solver")
print(result)

from qiskit_optimization import QuadraticProgram
from qiskit.providers.aer import QasmSimulator
from qiskit.algorithms.optimizers import SPSA
from qiskit.circuit.library import TwoLocal
from qiskit.utils import QuantumInstance
from qiskit.algorithms import VQE
from qiskit_optimization.algorithms import MinimumEigenOptimizer
seed = 1234
qp = QuadraticProgram()
qp.from_docplex(mdl)
backend = QasmSimulator()
optimizer = SPSA(maxiter=100)
ry = TwoLocal(2, 'ry', 'cz', reps=3, entanglement='full')
quantum_instance = QuantumInstance(backend=backend, seed_simulator=seed, seed_transpiler=seed)
vqe_mes = VQE(ry, optimizer=optimizer, quantum_instance=quantum_instance)
vqe = MinimumEigenOptimizer(vqe_mes)
result = vqe.solve(qp)
print("result of VQE solver")
print(result)
print(result.raw_samples)
```

output:

```
\ This file has been generated by D0cplex
\ ENCODING=ISO-8859-1
\Problem name: docplex model

Minimize
  obj: x - 2 y
Subject To

Bounds
  0 <= x <= 1
  0 <= y <= 1

Binaries
  x y
End

result of CPLEX solver
solution for: docplex model
objective: -2
y=1

result of VQE solver
optimal function value: 1.0
optimal value: [1. 0.]
status: SUCCESS
```

Assignees

- a-matsuo
- t-imamichi

Labels

priority: critical type: bug

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Fix bit ordering and probabilities of samples in optimization_algorithm.py
qiskit-community/qiskit-optimization

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



What is the expected behavior?

The solution of VQE should be `[0. 1.]`.

Suggested solutions

We may need to reverse the bitstrings as `x = np.fromiter(list(bitstr[::-1]), dtype=int)`. But, it may affect other algorithms and unit tests.

[https://github.com/Qiskit/qiskit-](https://github.com/Qiskit/qiskit-optimization/blob/54a6d0750492132aa019975e3a8648c507b7553c/qiskit_optimization/algorithms/optimization_algorithm.py#L511)

[optimization/blob/54a6d0750492132aa019975e3a8648c507b7553c/qiskit_optimization/algorithms/optimization_algorithm.py#L511](https://github.com/Qiskit/qiskit-optimization/blob/54a6d0750492132aa019975e3a8648c507b7553c/qiskit_optimization/algorithms/optimization_algorithm.py#L511)



t-imamichi assigned [t-imamichi](#) and [a-matsuo](#) on Apr 21, 2021



t-imamichi added `priority: critical` `type: bug` on Apr 21, 2021



a-matsuo mentioned this on Apr 22, 2021

[Fix bit ordering and probabilities of samples in optimization_algorithm.py #97](#)



Cryoris on May 7, 2021

Contributor ...

Do we need to reverse the solution bitstring or would it maybe be better to reverse the bits in the Hamiltonian? Then the solution with the `MinimumEigenOptimizer` would be consistent with both CPLEX and VQE.

If we only flip the solution in `MinimumEigenOptimizer`, the solution bitstring will be the same as for CPLEX but if I do a manual optimization by using VQE directly, the order will not be the same, right?



t-imamichi on May 7, 2021

Collaborator Author ...

Yes, it will be fixed in [#97](#)



a-matsuo on May 7, 2021

Contributor ...

If we only flip the solution in `MinimumEigenOptimizer`, the solution bitstring will be the same as for CPLEX but if I do a manual optimization by using VQE directly, the order will not be the same, right?

You're right. I only flipped the solution in `MinimumEigenOptimizer` in [#97](#).

Hmm, ya, it might be better to reverse the bits in the Hamiltonian for consistency.

We can reverse the bits in the Hamiltonian, easily. Currently, variable `x_0` is mapped to qubit `q_0`, `x_1` -> `q_1`, ..., and `x_n` -> `q_n`. If we want reverse the bits in the Hamiltonian, we map `x_0` -> `q_n`, `x_1` -> `q_n-1`, ..., and `x_n` -> `q_0`.



t-imamichi on May 7, 2021

Collaborator Author ...

Oh, sorry. I missed the point. We need to revisit the ordering of Hamiltonian. How about discussing it in another issue?





t-imamichi on May 7, 2021

Collaborator Author ...

Oh, sorry. I missed the point. We need to revisit the ordering of Hamiltonian. How about discussing it in another issue?



Cryoris on May 7, 2021 · edited by Cryoris

Edits Contributor ...

Why not in this issue (since the bug is described here)? 🤔 If the changes are not too big it would probably be better to do only one fix instead of first re-ordering the solution bits and then undoing this by re-ordering the Hamiltonian.

@a-matsuo do you think you could check if reversing the Hamiltonian order fixes the problem consistently? 🤔



t-imamichi on May 7, 2021

Collaborator Author ...

It's just in case. Some users may use the current order.



t-imamichi on May 7, 2021

Collaborator Author ...

But, as you say, it's a good opportunity to change the order because this library is new.



t-imamichi on May 7, 2021

Collaborator Author ...

Anyways, we need to clarify the order of Hamiltonian in docstrings of `to_ising` and `from_ising`.



a-matsuo on May 7, 2021

Contributor ...

@Cryoris Sure. Let me check reversing the Hamiltonian order works correctly as I expect. I will work on it next week.

@t-imamichi Ya, we need to clarify the order of Hamiltonian in docstrings if we change it.



1



t-imamichi on May 7, 2021 · edited by t-imamichi

Edits Collaborator Author ...

If we reverse the Hamiltonian ordering, `QuantumInstance.initial_layout` might be mapped in the reverse order. We need to verify it. Users expect `initial_layout` is mapped to the same order of variables. `coupling_map` may have the same problem.



t-imamichi on May 11, 2021

Collaborator Author ...

I'm checking some PRs related to endian and wondering what is the best option.

[Qiskit/qiskit#5848](#)

[Qiskit/qiskit#4849](#)

@ikkoHam Do you have any idea from `PauliSumOp` point of view?





ikkoham on May 11, 2021

Contributor ...

I think all of the qiskit operators (including PauliSumOp) and circuits should be little endian if possible.
[Qiskit/qiskit#1148](#)

Second quantized operators in qiskit-nature is only exception since the application users are familiar with BIG endian.



t-imamichi on May 11, 2021

Collaborator Author ...

Thanks @ikkoham. The bit ordering is based on the Terra's convention. So, I don't think we need to reverse it in this optimization layer. The same mapping (variable_i to qubit_i) makes sense to me. I would like to merge [#97](#) as is.



a-matsuo on May 11, 2021 · edited by a-matsuo

Edits Contributor ...

From the perspective of operators, when users manually make Ising Hamiltonians for optimization problems, it is natural to think mapping qubit 1 to variable 1. I think the current way (mapping x1 to q1) makes sense.

If we only flip the solution in MinimumEigenOptimizer, the solution bitstring will be the same as for CPLEX but if I do a manual optimization by using VQE directly, the order will not be the same, right?

Then, for the above issue, maybe, writing clear docstrings is the only solution?



t-imamichi on May 11, 2021

Collaborator Author ...

Agree. It's good to add a docstring about mapping of variables to qubits.



Cryoris on May 11, 2021

Contributor ...

I don't think it's a good long term solution to have different qubit ordering in Qiskit Core and in the application modules. This will require users to exactly know in which part of Qiskit they are operating and swap qubit orders in the right places -- which can even be confusing for us developers! 😞

It's true that many users are used to big endian notation, but Qiskit just doesn't use big endian so I would argue it's better to be clear about this and not have different conventions in on software package. As a compromise, we could add a `little_endian` argument in some places, which is `True` by default but can be set to `False` such that users can use big endian.



t-imamichi on May 11, 2021 · edited by t-imamichi

Edits Contributor Author ...

Yes. So, let's keep the current qubit order (variable_i -> qubit_i) to align with qiskit core's convention and clarify it in the docstring of `to_ising` and `from_ising`.

The endian option sounds interesting. I would like to discuss it as a future work.



t-imamichi on May 12, 2021

Collaborator Author ...

I updated the docstrings of `to_ising` and `from_ising`. If there is no more comments about [#97](#), I will merge it.





t-imamichi on May 11, 2021

Collaborator Author ...

Agree. It's good to add a docstring about mapping of variables to qubits.



Cryoris on May 11, 2021

Contributor ...

I don't think it's a good long term solution to have different qubit ordering in Qiskit Core and in the application modules. This will require users to exactly know in which part of Qiskit they are operating and swap qubit orders in the right places -- which can even be confusing for us developers! 😞

It's true that many users are used to big endian notation, but Qiskit just doesn't use big endian so I would argue it's better to be clear about this and not have different conventions in on software package. As a compromise, we could add a `little_endian` argument in some places, which is `True` by default but can be set to `False` such that users can use big endian.



t-imamichi on May 11, 2021 · edited by t-imamichi

Edits ▾ Collaborator Author ...

Yes. So, let's keep the current qubit order (`variable_i` -> `qubit_i`) to align with qiskit core's convention and clarify it in the docstring of `to_ising` and `from_ising`.

The endian option sounds interesting. I would like to discuss it as a future work.



t-imamichi on May 12, 2021

Collaborator Author ...

I updated the docstrings of `to_ising` and `from_ising`. If there is no more comments about [#97](#), I will merge it.



Cryoris on May 12, 2021

Contributor ...

Ok 👍 yeah let's revisit the ordering in another issue, I'm not sure I fully understood where which order is assumed 😊 Thanks for the clarifications!



1

🔒 t-imamichi closed this as **completed** in [#97](#) on May 13, 2021

🔄 t-imamichi reopened this on May 19, 2021

🔒 t-imamichi closed this as **completed** on May 19, 2021



Add a comment